AULA 2

Por dentro do sistema de arquivos

Inodes, alocação, cache, falhas e recuperação.

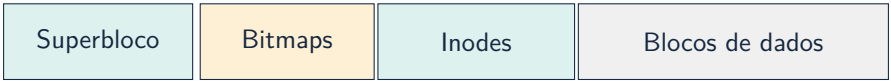
Retomada da Aula 1

- Vimos arquivos como sequência de bytes e diretórios como organização de nomes.
- O descritor de arquivo permite acesso por chamadas POSIX.
- `lseek()` altera a posição; `read()` e `write()` transferem dados.
- Agora veremos **como os bytes são organizados** e **o que ocorre em uma falha**.

Sistemas de arquivos: responsabilidades

- Associar nomes a objetos persistentes.
- Organizar dados e metadados em estruturas de armazenamento.
- Gerenciar blocos livres e ocupados.
- Oferecer operações de leitura, escrita, criação e remoção.
- Buscar consistência e recuperação após falhas.

Uma visão simplificada de disco

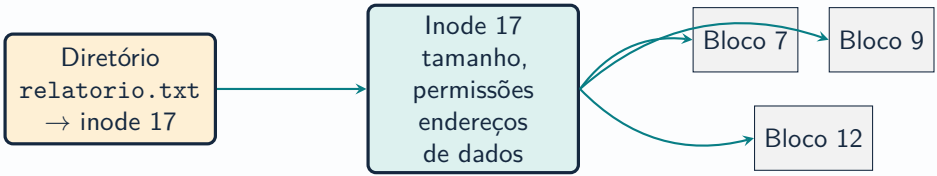


Esquema didático: a disposição real depende do sistema de arquivos.

Bloco e metadados

- **Bloco:** unidade lógica de alocação/armazenamento, por exemplo 4 KiB.
- **Superbloco:** descreve características e parâmetros do sistema de arquivos.
- **Bitmap:** informa quais unidades estão livres ou ocupadas.
- **Inode:** guarda metadados e informações para localizar o conteúdo.
- **Diretório:** relaciona nomes a objetos identificados por inodes.

Nome, inode e blocos



O nome está em um diretório; os dados do arquivo não ficam armazenados dentro do nome.

O que guarda um inode?

- Identificador do proprietário e grupo; permissões.
- Tamanho do arquivo e tempos registrados pelo sistema de arquivos.
- Quantidade de links físicos associados ao inode.
- Informações necessárias para localizar os blocos de dados.
- **Não guarda, em geral, o nome do arquivo**: esse vínculo pertence ao diretório.

Links físicos e simbólicos

- **Hard link:** outra entrada de diretório para o mesmo inode no mesmo sistema de arquivos.
- **Link simbólico:** arquivo especial que contém um caminho para outro objeto.
- Links físicos compartilham conteúdo e inode; links simbólicos têm inode próprio.
- Remover um nome não apaga automaticamente os dados enquanto existirem outras referências relevantes.
- Um link simbólico pode ficar quebrado se o destino deixar de existir.

Código 03: stat e lstat

```c
/* Trechos de 03_metadados.c */
struct stat info, destino;
lstat(nome, &info); /* examina o proprio caminho */
stat(nome, &destino); /* segue link simbolico */

printf("inode do caminho: %llu\n",
       (unsigned long long)info.st_ino);
printf("inode do destino: %llu\n",
       (unsigned long long)destino.st_ino);
```

O programa completo também mostra tipo, tamanho, permissões e número de links físicos.

DEMONSTRACAO 03

Acompanhar no terminal

Diretório: `SO/05_persistencia/exemplos`

```
./01_arquivo gravar
./03_metadados registro.txt
ln -f registro.txt link_fisico.txt
ln -sfn registro.txt link_simbolico.txt
./03_metadados link_fisico.txt
./03_metadados link_simbolico.txt
ls -li registro.txt link_*.txt
```

Observar: `registro.txt` e `link_fisico.txt` têm mesmo inode; o link simbólico tem inode próprio. Os comandos com exttt-f podem ser repetidos sem erro de arquivo existente.

Para discutir com a turma

1. Um *hard link* cria uma cópia independente dos bytes?
2. O que muda no contador de links físicos após executar `ln`?
3. Por que `stat()` e `lstat()` podem retornar inodes diferentes para o mesmo caminho?

Responda primeiro; confira no código ou no terminal em seguida.

Alocação e blocos livres

- Um arquivo pequeno pode ocupar um bloco de dados mesmo que use poucos bytes.
- O sistema de arquivos precisa localizar espaço livre para novos blocos.
- Bitmaps e outras estruturas acompanham a ocupação.
- A alocação de dados e de metadados exige atualizações consistentes.
- Fragmentação e localidade influenciam o custo de leitura e escrita.

Leitura e escrita: passos essenciais

Leitura

- Localizar o nome no diretório.
- Encontrar o inode e os blocos do arquivo.
- Consultar o cache; buscar blocos quando necessário.
- Copiar bytes para o processo.

Escrita

- Encontrar ou alocar blocos.
- Atualizar conteúdo e metadados.
- Registrar modificações no cache e programar E/S.
- Pedir sincronização quando a aplicação precisar de durabilidade.

As etapas são um modelo conceitual; implementações reais podem combinar e otimizar operações.

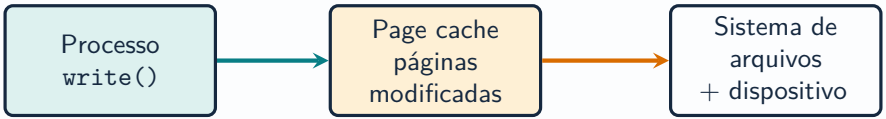
Localidade e fragmentação

- **Localidade espacial:** blocos próximos tendem a reduzir deslocamentos/consultas.
- **Localidade temporal:** dados acessados recentemente têm chance de ser reutilizados.
- Fragmentação pode espalhar partes de um arquivo em diferentes regiões.
- No HDD, localização física é especialmente relevante; em SSDs, outras políticas internas importam.
- O cache também altera o desempenho percebido pelo programa.

Page cache: por que existe?

- O kernel pode guardar páginas de arquivos em RAM para acelerar acessos.
- **Leitura:** uma próxima leitura pode ser satisfeita a partir do cache.
- **Escrita:** páginas modificadas (*dirty*) podem aguardar gravação posterior.
- O retorno de `write()` normalmente significa que os dados foram aceitos, não que chegaram à mídia.
- O comportamento depende do sistema de arquivos, da montagem e do dispositivo.

write, cache e dispositivo



Uma chamada de escrita pode retornar antes de ocorrer a gravação física.

write, close, fsync: papéis diferentes

- `write(fd,...)`: solicita a escrita de bytes; retorna quantidade ou erro.
- `close(fd)`: fecha o descritor; não é sinônimo de sincronização durável.
- `fsync(fd)`: solicita que dados modificados e metadados relevantes sejam sincronizados.
- `fsync()` pode falhar e seu resultado deve ser verificado.
- Para durabilidade de criação/renomeação de nomes, pode ser preciso sincronizar também o diretório.

Código 04: usando fsync

```c
/* Trechos de 04_fsync.c */
int fd = open("sincronizado.txt",
              O_CREAT | O_WRONLY | O_APPEND, 0644);
write(fd, texto, sizeof(texto) - 1);
puts("1. write() concluido");
fsync(fd);
puts("2. fsync() concluido");
close(fd);
```

O código completo testa os retornos de `open`, `write`, `fsync` e `close`.

DEMONSTRACAO 04

Acompanhar no terminal

Diretório: `SO/05_persistencia/exemplos`

```
./04_fsync
cat sincronizado.txt
./04_fsync
wc -l sincronizado.txt
```

Observar: Compare a ordem das mensagens. O programa demonstra a chamada, mas não simula queda de energia nem prova durabilidade sob qualquer falha.

Para discutir com a turma

1. Se `write()` retornou sucesso, os bytes necessariamente já estão gravados no SSD?
2. Por que o resultado da demonstração não prova resistência a uma falta de energia?
3. Qual a diferença entre persistir o conteúdo de um arquivo e persistir uma nova entrada de diretório?

Responda primeiro; confira no código ou no terminal em seguida.

Consistência após falhas: o problema

- Uma operação lógica pode exigir atualizar vários blocos de metadados e de dados.
- Uma queda de energia pode ocorrer **entre duas dessas atualizações**.
- Exemplo: bloco marcado como ocupado, mas inode ainda sem referência a ele.
- O sistema de arquivos precisa evitar ou reparar inconsistências estruturais.
- Consistência dos metadados e durabilidade dos dados recentes são questões diferentes.

Experimento mental: criar um arquivo

Para discutir com a turma

1. O sistema precisa criar uma entrada de diretório e um inode. Se só uma atualização chegar ao disco, o que pode acontecer?
2. Quais informações precisam concordar após uma falha?
3. Por que atualizar estruturas **em etapas** cria um problema de consistência?

Responda primeiro; confira no código ou no terminal em seguida.

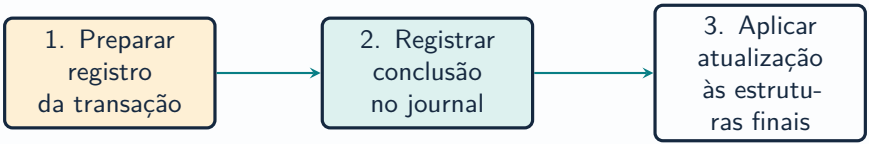
fsck: verificação e reparo

- Ferramentas do tipo `fsck` procuram inconsistências na estrutura de sistemas de arquivos.
- Exemplos: blocos indevidamente compartilhados, contadores incorretos e referências inválidas.
- Alguns problemas podem ser reparados; outros podem envolver perda de informação.
- A verificação completa pode exigir tempo proporcional ao tamanho/estrutura do sistema de arquivos.
- **Não executar fsck de escrita** em partição montada em uso sem seguir as recomendações específicas.

Journaling: ideia principal

- Antes de aplicar certas atualizações, o sistema registra uma transação em um **journal**.
- Após uma falha, o journal ajuda a identificar quais transações precisam ser recuperadas.
- Assim, o sistema procura manter estruturas do sistema de arquivos consistentes.
- O journal pode tratar metadados e dados de formas diferentes, segundo a implementação e o modo de operação.
- Journaling **não substitui** backups nem garante preservação de todo dado recente.

O princípio de uma transação em journal



Esquema simplificado de escrita antecipada: detalhes variam entre sistemas e modos de journaling.

Consistência não é sinônimo de backup

Journaling / fsck

- Visam manter ou recuperar estruturas coerentes.
- Podem lidar com falhas e interrupções de atualizações.
- Não recuperam necessariamente dados apagados pelo usuário.

Backup

- Mantém cópias recuperáveis em outro local/meio.
- Ajuda diante de exclusão acidental, corrupção ou desastre.
- Exige política de versões, testes e proteção da cópia.

ext4: exemplo de sistema de arquivos do Linux

- Utiliza inodes, diretórios e estruturas de alocação de blocos.
- Suporta journaling e extensões (*extents*) para mapear blocos.
- Permite observar metadados com ferramentas usuais do Linux.
- Os detalhes reais de alocação e escrita dependem de versão, opções de montagem e estado do sistema.
- Nosso modelo de inode é conceitual; a implementação do ext4 possui detalhes adicionais.

Terminal: observar o sistema de arquivos

```
findmnt -T .
lsblk -f
df -h .
du -sh .
stat registro.txt
```

Esses comandos consultam informações, sem modificar a partição. `df` e `du` medem coisas distintas.

Integridade e proteção

Erros que podem ocorrer

- Falhas de dispositivo e setores problemáticos.
- Corrupção silenciosa de dados.
- Erro humano e exclusão acidental.
- Perda de alimentação durante atualização.

Mecanismos de defesa

- Checksums e verificação de integridade.
- Redundância entre dispositivos.
- Backups e versões recuperáveis.
- Ferramentas de verificação e recuperação.

RAID: por que combinar discos?

- **RAID 0:** distribui dados entre discos; melhora paralelismo, sem redundância.
- **RAID 1:** espelha dados; uma cópia pode atender em caso de falha da outra.
- **RAID 5:** distribui blocos e paridade; tolera a falha de um disco no arranjo usual.
- O resultado depende da configuração, do controlador/software e do modo de falha.
- **RAID não é backup**: exclusões e corrupções lógicas podem se propagar.

Comparação: três camadas de proteção

Para discutir com a turma

1. Journaling ajuda contra quais tipos de inconsistência?
2. RAID 1 ajuda quando um disco falha. Ele restaura um arquivo que o usuário apagou?
3. Qual medida ajuda a recuperar um arquivo excluído há duas semanas?
4. Que papel `fsync()` desempenha para um programa que grava dados importantes?

Responda primeiro; confira no código ou no terminal em seguida.

Síntese final: persistência

- **Interface:** descritores, `open`, `read`, `write`, `lseek`, `close`.
- **Estruturas:** diretórios, inodes, blocos e alocação.
- **Desempenho:** HDD/SSD, localidade e page cache.
- **Confiabilidade:** `fsync`, journaling, `fsck`, integridade e backup.
- **Ideia-chave:** abstrair dados persistentes envolve armazenamento, organização e recuperação.

Bibliografia e leitura orientada — OSTEP 1.10

- **Cap. 36 — I/O Devices:** controladores, polling, interrupções e DMA.
- **Cap. 37 — Hard Disk Drives; cap. 44 — Flash-based SSDs:** dispositivos.
- **Cap. 39 — Files and Directories:** interface; **cap. 40 — File System Implementation:** estrutura.
- **Cap. 42 — FSCK and Journaling:** consistência; **cap. 38 — Redundant Disk Arrays (RAID)**.
- **Cap. 45 — Data Integrity and Protection:** integridade de dados.

Ideia-chave: Fonte: Arpaci-Dusseau e Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v. 1.10 (2023).

Todos os arquivos desta aula:

`SO/05_persistencia/`

- `slides/apresentacao.tex` e `apresentacao.pdf`.
- `exemplos/01_arquivo.c`, `02_lseek.c`, `03_metadados.c`, `04_fsync.c`.
- `exemplos/Makefile` e `README.md` com comandos de execução.

Leitura, teste, hipótese e explicação: a mesma abordagem das aulas anteriores.